Virtual Maize Field — Setup & Configuration Guide

Ignition Fortress + ROS 2 Humble

April 2026

Contents

1. Overview **3**

2. Prerequisites **3**
2.1 ROS 2 Humble . . . 3
2.2 Ignition Fortress . . . 3
2.3 ROS-Ignition Bridge Packages . . . 3
2.4 Python Dependencies . . . 4
2.5 Colcon Build Tool . . . 4

3. Installation **4**
3.1 Clone the Repository . . . 4
3.2 Build the Workspace . . . 4
3.3 Source the Workspace . . . 4

4. Usage **4**
4.1 Generate a World . . . 4
4.2 Launch the Simulation . . . 5
4.3 Spawn a Robot . . . 5
4.4 Convenience Script . . . 6

5. Configuration **6**
5.1 Config File Format . . . 6
5.2 Creating Additional Configs . . . 6
5.3 Using Pre-built Competition Configs . . . 6

6. Complete Parameter Reference **8**
6.1 Field Layout . . . 8
6.2 Row Segment Types . . . 8
6.3 Straight Segment Parameters . . . 8
6.4 Sinusoidal Curve Parameters . . . 9
6.5 Curved Segment Parameters . . . 9
6.6 Island Segment Parameters . . . 9
6.7 Ground / Terrain . . . 9
6.8 Plant Properties . . . 10

6.9 Holes / Gaps in Rows . . . 10
6.10 Crop Types . . . 11
6.11 Weeds . . . 11
6.12 Litter / Obstacles . . . 12
6.13 Special Objects . . . 12
6.14 Reproducibility . . . 13

7. Example Configurations **14**
7.1 Simple Straight Field (Testing) . . . 14
7.2 Large Dense Field . . . 14
7.3 Challenging Navigation Course . . . 14
7.4 Vision-Only (No Collisions) . . . 14

8. Troubleshooting **16**
Simulation won't start . . . 16
Config changes not taking effect . . . 16
"Config file not found" error . . . 16
Matplotlib / minimap error . . . 16
Ignition Fortress GUI is slow . . . 16

9. File Structure **16**

1. Overview

The **virtual_maize_field** package generates randomized corn (maize) field environments for the Gazebo simulator. It is maintained by Wageningen University & Research, Kamaro Engineering, and the University of Hohenheim.

Repository: `https://github.com/FieldRobotEvent/virtual_maize_field`

Our setup uses:

Component	Version
OS	Ubuntu 22.04 (Jammy)
ROS	ROS 2 Humble
Simulator	Ignition Fortress
Branch	`ros2-ign`

Workspace location: `~/het_project/next_gen_lab/sim_corn_fields/`

2. Prerequisites

Before using the simulation, ensure the following are installed on your system.

2.1 ROS 2 Humble

```
# Verify installation
source /opt/ros/humble/setup.bash
echo $ROS_DISTRO    # Should print: humble
```

If not installed, follow the official ROS 2 Humble installation guide for Ubuntu 22.04.

2.2 Ignition Fortress

```
# Add the Gazebo package repository
sudo wget https://packages.osrfoundation.org/gazebo.gpg \
    -O /usr/share/keyrings/pkgs-osrf-archive-keyring.gpg

echo "deb [arch=$(dpkg --print-architecture) \
    signed-by=/usr/share/keyrings/pkgs-osrf-archive-keyring.gpg] \
    http://packages.osrfoundation.org/gazebo/ubuntu-stable \
    $(lsb_release -cs) main" | \
    sudo tee /etc/apt/sources.list.d/gazebo-stable.list > /dev/null

# Install Ignition Fortress
sudo apt-get update
sudo apt-get install -y ignition-fortress
```

2.3 ROS-Ignition Bridge Packages

These allow ROS 2 topics to communicate with Ignition Gazebo.

```
sudo apt-get install -y \
    ros-humble-ros-ign \
    ros-humble-ros-ign-bridge \
    ros-humble-ros-ign-gazebo \
    ros-humble-ros-ign-image
```

2.4 Python Dependencies

The world generator requires `numpy`, `matplotlib`, and `pyyaml`. These are typically included with ROS 2 Humble, but verify:

```
pip3 install numpy matplotlib pyyaml
```

2.5 Colcon Build Tool

```
pip3 install colcon-common-extensions
```

3. Installation

3.1 Clone the Repository

```
cd ~/het_project/next_gen_lab/sim_corn_fields/
mkdir -p src
cd src
git clone -b ros2-ign \
    https://github.com/FieldRobotEvent/virtual_maize_field.git
```

3.2 Build the Workspace

```
cd ~/het_project/next_gen_lab/sim_corn_fields/
source /opt/ros/humble/setup.bash
colcon build
```

3.3 Source the Workspace

Run this in every new terminal, or add it to your `~/.bashrc`:

```
source /opt/ros/humble/setup.bash
source ~/het_project/next_gen_lab/sim_corn_fields/install/setup.bash
```

4. Usage

4.1 Generate a World

Generate a corn field world file using default parameters:

```
ros2 run virtual_maize_field generate_world
```

Generate from a named config file (stored in `config/` folder):

```
ros2 run virtual_maize_field generate_world custom_field
```

Generate with individual parameter overrides:

```
ros2 run virtual_maize_field generate_world \
    --rows_count 10 \
    --row_length 15.0 \
    --weeds 5 \
    --plant_height_min 0.4 \
    --plant_height_max 0.8
```

Show the 2D ground truth map after generation:

```
ros2 run virtual_maize_field generate_world custom_field --show_map
```

Output files are saved to `~/.ros/virtual_maize_field/`:

File	Description
`generated.world`	The SDF world file for Ignition Gazebo
`gt_map.png`	2D ground truth minimap image
`gt_map.csv`	Ground truth plant locations (CSV)
`markers.csv`	Location marker coordinates (if enabled)
`robot_spawner.launch.py`	Launch file to spawn a robot at the start pose
`virtual_maize_field_heightmap.png`	Terrain heightmap image

4.2 Launch the Simulation

```
ros2 launch virtual_maize_field simulation.launch.py
```

Launch arguments:

Argument	Default	Description
`use_sim_time`	`True`	Use simulation clock for ROS time
`paused`	`False`	Start simulation in paused state
`headless`	`False`	Run without GUI (server only)
`world_path`	`~/.ros/virtual_maize_field/`	Path to world files
`world_name`	`generated.world`	Name of the world file

Example — headless mode:

```
ros2 launch virtual_maize_field simulation.launch.py headless:=True
```

4.3 Spawn a Robot

After the simulation is running, use the generated spawner launch file:

```
ros2 launch ~/.ros/virtual_maize_field/robot_spawner.launch.py \
    robot_description_path:=/path/to/your/robot.urdf
```

4.4 Convenience Script

A helper script `run_field.sh` is provided in the workspace root:

```
cd ~/het_project/next_gen_lab/sim_corn_fields/

./run_field.sh                    # Generate + launch (custom_field config)
./run_field.sh my_config          # Use a different config
./run_field.sh --headless         # No GUI
./run_field.sh --map              # Show 2D map after generation
./run_field.sh --generate-only   # Only generate, don't launch sim
./run_field.sh --help             # Show help and list all configs
```

The script automatically sources ROS 2 and the workspace, kills any running simulation, generates the world, and launches Ignition Fortress.

5. Configuration

5.1 Config File Format

Config files are YAML files stored in `src/virtual_maize_field/config/`. You only need to include parameters you want to override; all others use their defaults.

Custom config location: `src/virtual_maize_field/config/custom_field.yaml`

Example minimal config:

```yaml
row_length: 15.0
rows_count: 10
row_segments:
  - straight
weeds: 5
seed: 42
```

After editing a config file, rebuild the workspace before using it:

```
colcon build
source install/setup.bash
```

5.2 Creating Additional Configs

Create a new YAML file in the `config/` folder:

```
# Example: create a dense field config
cp src/virtual_maize_field/config/custom_field.yaml \
   src/virtual_maize_field/config/dense_field.yaml
# Edit dense_field.yaml as needed
colcon build
ros2 run virtual_maize_field generate_world dense_field
```

5.3 Using Pre-built Competition Configs

The package includes 18 configs from the Field Robot Event competitions:

Config Name	Description
`fre21_task_1`	FRE 2021 Task 1 — straight rows
`fre21_task_2`	FRE 2021 Task 2 — straight rows, 11 rows
`fre21_task_3`	FRE 2021 Task 3 — weeds and litter
`fre21_task_4`	FRE 2021 Task 4 — location markers
`fre22_task_navigation`	FRE 2022 Navigation — sinusoidal rows
`fre22_task_mapping`	FRE 2022 Mapping — requires extra models
`*_mini`	Smaller version (fewer rows, shorter)
`*_fast`	Lightweight version (for quick testing)

Usage:

```
ros2 run virtual_maize_field generate_world fre22_task_navigation
```

6. Complete Parameter Reference

All parameters that can be set in a config YAML file or passed as command-line arguments.

6.1 Field Layout

Parameter	Type	Default	Description
`row_length`	float	12.0	Total length of each crop row in meters. Longer rows create a larger field.
`rows_count`	int	6	Number of parallel crop rows in the field.
`row_width`	float	0.75	Spacing between adjacent rows in meters. This is the gap your robot must navigate through.
`rows_curve_budget`	float	1.5708	Maximum total curvature allowed across all row segments, in radians. Default is $\pi/2$ (90 degrees). Higher values allow more winding rows.

6.2 Row Segment Types

The `row_segments` parameter accepts a list of segment types. The generator randomly chains segments from this list to form each row.

Segment Type	Description
`straight`	A straight-line segment. Length is randomly chosen between `straight_length_min` and `straight_length_max`.
`curved`	A circular arc segment. Radius and arc angle are randomly chosen from their respective min/max ranges.
`sincurved`	A sinusoidal (S-curve) segment. Creates smooth, wave-like row patterns.
`island`	A circular detour around an obstacle. Creates a loop-around pattern in the row.

Config example:

```yaml
row_segments:
  - straight
  - curved
  - sincurved
```

6.3 Straight Segment Parameters

Parameter	Type	Default	Description
`row_segment_straight_length_min`	float	0.5	Minimum length of a straight segment (meters).
`row_segment_straight_length_max`	float	1.0	Maximum length of a straight segment (meters).

6.4 Sinusoidal Curve Parameters

Parameter	Type	Default	Description
`row_segment_sincurved_offset_min`	float	0.5	Minimum lateral offset of the sine wave (meters). Controls how far the row deviates sideways.
`row_segment_sincurved_offset_max`	float	1.5	Maximum lateral offset of the sine wave (meters).
`row_segment_sincurved_length_min`	float	3.0	Minimum length of one sine wave period (meters).
`row_segment_sincurved_length_max`	float	5.0	Maximum length of one sine wave period (meters).

6.5 Curved Segment Parameters

Parameter	Type	Default	Description
`row_segment_curved_radius_min`	float	3.0	Minimum turning radius of the arc (meters). Smaller values create tighter curves.
`row_segment_curved_radius_max`	float	10.0	Maximum turning radius of the arc (meters).
`row_segment_curved_arc_measure_min`	float	0.3	Minimum arc angle (radians). 0.3 rad $\approx$ 17 degrees.
`row_segment_curved_arc_measure_max`	float	1.0	Maximum arc angle (radians). 1.0 rad $\approx$ 57 degrees.

6.6 Island Segment Parameters

Parameter	Type	Default	Description
`row_segment_island_radius_min`	float	1.0	Minimum radius of the island detour (meters).
`row_segment_island_radius_max`	float	3.0	Maximum radius of the island detour (meters).

6.7 Ground / Terrain

Parameter	Type	Default	Description
`ground_resolution`	float	0.02	Heightmap resolution in meters per pixel. Lower values produce more detailed terrain but increase file size.
`ground_elevation_max`	float	0.2	Maximum ground elevation variation in meters. Set to 0 for a perfectly flat field.
`ground_headland`	float	2.0	Width of flat area surrounding the crop rows (meters). Provides space for the robot to turn at row ends.
`ground_ditch_depth`	float	0.3	Depth of drainage ditches at the field boundary (meters). Set to 0 to disable ditches.

6.8 Plant Properties

Parameter	Type	Default	Description
`plant_spacing_min`	float	0.13	Minimum distance between consecutive plants in a row (meters).
`plant_spacing_max`	float	0.19	Maximum distance between consecutive plants in a row (meters). Actual spacing is randomized between min and max.
`plant_height_min`	float	0.3	Minimum plant height (meters). Plants are scaled randomly between min and max height.
`plant_height_max`	float	0.6	Maximum plant height (meters). Set both min and max equal for uniform height.
`plant_radius`	float	0.3	Base collision radius of each plant (meters). Used for physics collision detection.
`plant_radius_noise`	float	0.05	Random variation added to the plant radius (meters).
`plant_placement_error_max`	float	0.02	Maximum lateral offset from the ideal row line (meters). Simulates imprecise planting.
`plant_mass`	float	0.3	Mass of each plant in kilograms. Affects physics when a robot collides with a plant.

6.9 Holes / Gaps in Rows

Holes simulate crop failure — sections of a row where plants are missing.

Parameter	Type	Default	Description
`hole_prob`	float or list	[0.06, 0.06, 0.04, 0.04, 0.0, 0.0]	Probability (0.0–1.0) that a hole begins at each plant position. Provide a single value for all rows, or a list with one value per row.
`hole_size_max`	int or list	[7, 5, 5, 3, 0, 0]	Maximum number of consecutive plants removed in a hole. Provide a single value or one per row.

Note: When using per-row lists, the list length must equal `rows_count`.

Example — no holes at all:

```yaml
hole_prob: 0.0
hole_size_max: 0
```

Example — heavy gaps in first 3 rows only (6 rows total):

```yaml
hole_prob: [0.15, 0.12, 0.10, 0.0, 0.0, 0.0]
hole_size_max: [10, 8, 6, 0, 0, 0]
```

6.10 Crop Types

Parameter	Type	Default	Description
`crop_types`	list	[maize_01, maize_02]	Which 3D maize models to use. The generator randomly picks from this list for each plant.

Available crop models:

Model Name	Description
`maize_01`	Maize plant variant 1 (default mesh)
`maize_02`	Maize plant variant 2 (alternate mesh)

6.11 Weeds

Parameter	Type	Default	Description
`weeds`	int	0	Number of weed objects to scatter randomly within the crop rows. Set to 0 to disable.

Parameter	Type	Default	Description
`weed_types`	list	[nettle, unknown_weed, dandelion]	Which weed models to use.

Available weed models:

Model Name	Description
`nettle`	Stinging nettle plant
`unknown_weed`	Generic weed plant
`dandelion`	Dandelion — includes multiple mesh variants

6.12 Litter / Obstacles

Parameter	Type	Default	Description
`litters`	int	0	Number of litter objects placed randomly in the field. Set to 0 to disable.
`litter_types`	list	[ale, beer, coke_can, retro_pepsi_can]	Which litter models to use.

Available litter models:

Model Name	Description
`ale`	Ale bottle (lying on its side)
`beer`	Beer bottle (lying on its side)
`coke_can`	Coca-Cola can
`retro_pepsi_can`	Retro-style Pepsi can

6.13 Special Objects

Parameter	Type	Default	Description
`ghost_objects`	bool	false	When true, all objects (plants, weeds, litter) have no collision geometry. Useful for vision-only tasks where you don't want the robot to physically interact with plants.
`location_markers`	bool	false	When true, places two marker poles (A and B) in the field. Used for navigation tasks where the robot must find specific locations. Marker positions are saved to `markers.csv`.

6.14 Reproducibility

Parameter	Type	Default	Description
`seed`	int	-1	Random seed for world generation. Use `-1` to generate a unique field each time (based on current timestamp). Set a fixed number (e.g., `42`) to reproduce the exact same field layout.
`load_from_file`	string	None	Path to a previously saved JSON world description file. Loads an exact field layout instead of generating a new one. Overrides all other parameters.

7. Example Configurations

7.1 Simple Straight Field (Testing)

```yaml
row_length: 8.0
rows_count: 4
row_segments:
  - straight
ground_elevation_max: 0.0
hole_prob: 0.0
hole_size_max: 0
seed: 42
```

7.2 Large Dense Field

```yaml
row_length: 20.0
rows_count: 14
row_width: 0.50
row_segments:
  - straight
plant_spacing_min: 0.10
plant_spacing_max: 0.14
plant_height_min: 0.5
plant_height_max: 0.9
seed: 100
```

7.3 Challenging Navigation Course

```yaml
row_length: 15.0
rows_count: 10
row_segments:
  - straight
  - curved
  - sincurved
rows_curve_budget: 2.0
ground_elevation_max: 0.35
hole_prob: 0.08
hole_size_max: 8
weeds: 10
litters: 5
location_markers: true
seed: 77
```

7.4 Vision-Only (No Collisions)

```yaml
row_length: 12.0
rows_count: 8
ghost_objects: true
weeds: 15
```

```yaml
litters: 8
location_markers: true
```

8. Troubleshooting

Simulation won't start

```
# Check if another Gazebo instance is running
pkill -f "ign gazebo"
# Retry
ros2 launch virtual_maize_field simulation.launch.py
```

Config changes not taking effect

Rebuild after editing any file in `src/`:

```
colcon build
source install/setup.bash
```

"Config file not found" error

Ensure the YAML file is in `src/virtual_maize_field/config/` and you rebuilt with `colcon build`.

Matplotlib / minimap error

If the ground truth map fails to save, this does not affect the world file itself. The simulation will still work. The issue is typically a matplotlib version incompatibility.

Ignition Fortress GUI is slow

Use headless mode for faster simulation when you don't need the 3D viewer:

```
ros2 launch virtual_maize_field simulation.launch.py headless:=True
```

Or use a `_fast` or `_mini` preset config for smaller fields.

9. File Structure

```
sim_corn_fields/
|-- run_field.sh                        # Convenience launch script
|-- docs/
|   |-- virtual_maize_field_guide.pdf   # This document
|-- src/
|   |-- virtual_maize_field/
|       |-- config/
|       |   |-- custom_field.yaml       # Your editable config
|       |   |-- fre21_task_*.yaml       # FRE 2021 competition configs
|       |   |-- fre22_task_*.yaml       # FRE 2022 competition configs
|       |-- launch/
|       |   |-- simulation.launch.py    # Main launch file
|       |-- models/                     # 3D models (maize, weeds, etc.)
|       |-- virtual_maize_field/
|           |-- world_generator/        # Python world generation code
```

```
|-- build/                                  # colcon build output
|-- install/                                # colcon install output
|-- log/                                    # Build and run logs
```